Experiment 1.8

Mapped Course Outcomes-CO3

CO3: Identify the connection establishment techniques and features
AIM:- Using Socket Programming implement the connection oriented service using standard ports in any programming language (C,C++, JAVA,Python etc).

Apparatus Required:-Pycharm

What will you learn:

In this lab, we will discuss the socket API and support for TCP communications between end hosts. Socket programming is the key API for programming distributed applications on the Internet.

We plan to learn the following:

- What is a socket?
- The client-server model
- Byte order
- TCP socket API
- Concurrent server design
- Example of echo client and iterative server
- Example of echo client and concurrent server

Prerequisites:-

1. A web browser with access to Pycharm

The basic Concepts

Program: A program is an executable file residing on a disk in a directory. A program is read into memory and is executed by the kernel as a result of an exec() function. The exec() has six variants, but we only consider the simplest one (exec()) in this course.

Process.: An executing instance of a program is called a process. Sometimes, task is used instead of process with the same meaning. UNIX guarantees that

every process has a unique identifier called the process ID. The process ID is always a non-negative integer.

File descriptors: File descriptors are normally small non-negative integers that the kernel uses to identify the files being accessed by a particular process. Whenever it opens an existing file or creates a new file, the kernel returns a file descriptor that is used to read or write the file. As we will see in this course, sockets are based on a very similar mechanism (socket descriptors).

The client-server model

· The client-server model is one of the most usedcommunication paradigms in networked systems.

· Clients normally communicates with one server at a time.

· From a server's perspective, at any point in time, it is notunusual for a server to be communicating with multiple clients. Client need toknow of the existence of and the address of the server, but the server does notneed to know the address of (or even the existence of) the client prior to theconnection being established

· Client and servers communicate by means of multiple layersof network protocols.

Client Code.

```
// A Java

program for

a Client

import

java.net.*;

import

java.io.*;



public class Client

{
```

```
    // initialize socket and
input output streams
private Socket socket
= null;     private
DataInputStream input  =
null;     private
DataOutputStream out    =
null;


    // constructor to put ip
address and port     public
Client(String address, int port)

    {
        // establish a connection
        try
        {
            socket = new Socket(address, port);
            System.out.println("Connected");


            // takes input from
terminal            input = new
DataInputStream(System.in);
```

```
            // sends output to the socket            out
= new
DataOutputStream(socket.getOutputStream());
        }
        catch(UnknownHostException u)
        {
            System.out.println(u);
        }
        catch(IOException i)
        {
            System.out.println(i);
        }
        // string to read message from input
        String line = "";

        // keep reading until
"Over" is input         while (!
line.equals("Over"))

{

t

r

y
```

```
            {
line =
input.readLine(
);
out.writeUTF(l
ine);
            }
            catch(IOException i)
            {
                System.out.println(i);
            }
        }
        // close the connection


try
{
inp
ut.c
lose
();
out.
clos
e();
soc
ket.
```

```
clos
e();
        }
        catch(IOException i)
        {
            System.out.println(i);
        }
    }
    public static void main(String args[])
    {
        Client client = new Client("127.0.0.1", 5000);
    }
}
```

Server

Code. // A

Java

program for

a Server

```
import
java.net.*;
import
java.io.*;
public class
Server
{
```

```
      //initialize  socket
and    input    stream
private           Socket
socket        =    null;
private   ServerSocket
server        =    null;
private
DataInputStream    in
= null;


   //
constructor
with port
public
Server(int
port)
   {
  // starts server and waits for a
              connection
      try
      {
          server = new ServerSocket(port);
          System.out.println("Server started");


          System.out.println("Waiting for a client ...");
```

```
            socket = server.accept();
            System.out.println("Client accepted");

            // takes input from the client socket
in = new DataInputStream(                new
BufferedInputStream(socket.getInputStream(
)));
            String line = "";
            // reads message from client until
"Over" is sent            while (!
line.equals("Over"))

{

t

r

y

                {
line =
in.readUTF();
                    System.out.println(line);
                }
                catch(IOException i)
                {
```

```
                System.out.println(i);
            }
        }
        System.out.println("Closing connection");
        // close
connection
socket.close();
in.close();
    }
    catch(IOException i)
    {
        System.out.println(i);
    }
  }
  public static void main(String args[])
  {
    Server server = new Server(5000);
  }
}
```

Related links and references

1. https://realpython.com/python-sockets/

2.https://docs.python.org/3/howto/sockets.html